🎥 Google Meet Video Recording Script (Side-by-Side Presentation Guide)

DISTRIBUTED URBAN AIR QUALITY ANALYTICS & FORECASTING (TRACK B) • NATURAL SPEAKING FLOW FOR SIR

Track B • HDFS → Spark → Spark MLlib

Teammate A (Driver / Storage Lead)

Shares screen, runs commands, presents Steps 1 & 2 (HDFS Data Lake) & Conclusion

Teammate B (Spark ETL & Analytics Lead)

Presents Steps 3 & 4 (Distributed Spark ETL, CPCB 16h validity & Diurnal Analytics)

Teammate C (Spark MLlib ML Lead)

Presents Steps 5 & 6 (Chronological 24h Forecasting +16.6% & Hazard Classification)

VIDEO INTRODUCTION (30 SECONDS) • TEAMMATE A (SCREEN SHARER)

*"Hello Sir. In this video, our team is presenting our Big Data project: **Distributed Urban Air Quality Analytics and AQI Forecasting**. We have built this project according to **Track B: Machine Learning at Scale**, implementing the complete distributed lifecycle: **Hadoop HDFS → Apache Spark distributed ETL → Spark MLlib** across Delhi NCR, Bengaluru, and Hyderabad adhering strictly to Indian **CPCB regulatory standards**. We have prepared numbered commands so we can run and demonstrate each stage cleanly. Let us begin with Step 1."*

🎬 Step 1: Inspecting Raw Telemetry in Hadoop HDFS

Speaker: Teammate A (Driver / Storage Lead)

🕹️ TERMINAL ACTION (WHAT DRIVER RUNS)

```
./1_hdfs_show_raw.sh
```

🖥️ SCREEN OUTPUT (WHAT SIR SEES)

```
$ hdfs dfs -ls -h /air_quality/raw/
Found 1 items
-rw-r--r--   1 yashsonawane supergroup     47.3 M 2026-10-14 23:13 /air_qualit
y/raw/raw_hourly_measurements.jsonl
```

🗣️ WHAT TO SAY TO SIR (NATURAL SPEAKING SCRIPT)

"Sir, as you can see on the screen, our command just listed our raw telemetry staged inside Hadoop HDFS under the path `/air_quality/raw/`.

You can see this file: **raw_hourly_measurements.jsonl**. It is **47.3 Megabytes** and contains **96,000 raw continuous hourly records** collected from the OpenAQ API across Delhi, Bengaluru, and Hyderabad.

We built a resumable client that respects OpenAQ's 60 req/min rate limit. Staging it in HDFS allows Hadoop to distribute the data with a 128 MB block size and 3x replication factor.

Now, let us look at Step 2 to see how we store the processed data."

Key Highlights to Point at:

- Point to **47.3 M**: Real continuous telemetry (96k rows), not dummy small data.
- Point to **/air_quality/raw/**: Demonstrates proper HDFS distributed file zoning.

🎬 Step 2: Inspecting Partitioned Parquet Lake in HDFS

Speaker: Teammate A (Driver / Storage Lead)

🕹️ TERMINAL ACTION (WHAT DRIVER RUNS)

```
./2_hdfs_show_parquet.sh
```

🖥️ SCREEN OUTPUT (WHAT SIR SEES)

```
$ hdfs dfs -ls -h /air_quality/processed/hourly_aqi_parquet/
Found 4 items
-rw-r--r--   1 yashsonawane supergroup          0 2026-10-15 02:22 /air_qualit
y/processed/hourly_aqi_parquet/_SUCCESS
drwxr-xr-x   - yashsonawane supergroup          0 2026-10-15 02:22 /air_qualit
y/processed/hourly_aqi_parquet/city=Bengaluru
drwxr-xr-x   - yashsonawane supergroup          0 2026-10-15 02:22 /air_qualit
y/processed/hourly_aqi_parquet/city=Delhi NCR
drwxr-xr-x   - yashsonawane supergroup          0 2026-10-15 02:22 /air_qualit
y/processed/hourly_aqi_parquet/city=Hyderabad
```

🗣️ WHAT TO SAY TO SIR (NATURAL SPEAKING SCRIPT)

"Sir, in Step 2, this is our processed data lake in HDFS.

Notice that instead of plain CSV files, we are using **columnar Apache Parquet with Snappy compression**, partitioned by city—`city=Bengaluru`, `city=Delhi NCR`, and `city=Hyderabad`.

This is crucial for performance because it enables **Partition Pruning**. When our downstream Spark queries ask for Delhi data, Spark reads only the Delhi directory and completely skips the other cities, saving disk I/O and network shuffle.

Now, my teammate will explain Step 3 and run our distributed Spark ETL."

Key Highlights to Point at:

- Point to **_SUCCESS**: Hadoop output committer validation mark.
- Point to **city=XXX folders**: Automatic Hive-style partitioning for fast parallel scans.

🎬 Step 3: Running Distributed Apache Spark ETL & CPCB Math

Speaker: Teammate B (Spark ETL & Analytics Lead)

🕹️ TERMINAL ACTION (WHAT DRIVER RUNS)

```
./3_spark_run_etl.sh
```

🖥️ SCREEN OUTPUT (WHAT SIR SEES)

```
$ python3 scripts/spark_etl.py
================ STARTING DISTRIBUTED SPARK ETL ================
Apache Spark Version: 4.2.0
[1/6] Ingesting raw dataset from: hdfs://localhost:9000/air_quality/raw/...
  Ingested 96,000 raw records.
[3/6] Normalizing timestamps to IST and harmonizing pollutant units...
[4/6] Reshaping and pivoting pollutants into hourly station records...
[5/6] Calculating rolling exposure windows and CPCB NAQI...
  Total Station-Hour Records Processed: 18,521
  Records Meeting Strict CPCB 16h NAQI Validity: 12,792

--- Sample Processed Station Records with CPCB AQI ---
+---------+----------------------+-------------------+------------------+----
----+---------------+
|city     |station_name          |ts_ist             |pm25_24h          |cpcb
_aqi|health_category|
+---------+----------------------+-------------------+------------------+----
----+---------------+
|Delhi NCR|R K Puram, Delhi - DPCC|2016-02-06 12:30:00|193.42857142857142|500.
0   |Severe         |
+---------+----------------------+-------------------+------------------+----
----+---------------+
Spark ETL Pipeline completed successfully in 20.71 seconds.
```

🗣️ WHAT TO SAY TO SIR (NATURAL SPEAKING SCRIPT)

(While running): "Sir, the driver just started our Apache Spark ETL script. Spark 4.2.0 is loading the 96,000 raw records from HDFS into an in-memory DataFrame."

(When output finishes): "As you can see, the pipeline completed in 20 seconds. Here is what happened:

1. In Step 3, we converted gases like NO2 and SO2 from parts-per-billion into µg/m³ using their molecular weights.
2. In Step 4, Spark pivoted individual pollutant streams into **18,521 station-hour records**.
3. In Step 5, we applied the official Indian **CPCB 16-hour completeness rule**: a 24-hour moving window is only valid if at least 16 hourly readings exist. Exactly **12,792 records** met this strict rule.
4. In the sample table, you can see calculated 24h PM2.5, PM10, and final AQI—R K Puram shows an AQI of 500, which falls into the **Severe** category.

Now let us run Step 4 to see our distributed Spark SQL analytics and generated charts."

🎬 Step 4: Distributed Spark SQL Analytics & Descriptive Stats

Speaker: Teammate B (Spark ETL & Analytics Lead)

🕹️ TERMINAL ACTION (WHAT DRIVER RUNS)

```
./4_spark_run_analytics.sh
```

🖥️ SCREEN OUTPUT (WHAT SIR SEES)

```
$ python3 scripts/run_analytics.py
================ RUNNING DISTRIBUTED ANALYTICS ================
[1/4] Calculating city-wise descriptive statistics...
     city  total_hours  valid_aqi_hours  avg_pm25  std_pm25  avg_pm10  avg_cpc
b_aqi  max_cpcb_aqi
Delhi NCR         8131             5924     121.9      97.7     288.8
382.0         500.0
Bengaluru         6126             3378      28.8      52.7      68.9
293.9         500.0
Hyderabad         4264             3490      28.0      21.6      69.7
265.1         500.0

[2/4] Computing CPCB health category distribution by city...
     city health_category  category_count pct_of_city
Delhi NCR          Severe            3701        62.5
Bengaluru    Satisfactory            1236        36.6
Hyderabad    Satisfactory            1440        41.3

[4/4] Rendering analytics visualizations...
  [COMMAND 4 - FIGURE 1] Saved: reports/figures/cmd4_fig1_diurnal_pm25_trends.
png
  [COMMAND 4 - FIGURE 2] Saved: reports/figures/cmd4_fig2_cpcb_category_distri
bution.png
  [COMMAND 4 - FIGURE 3] Saved: reports/figures/cmd4_fig3_city_aqi_boxplots.pn
g
```

🗣️ WHAT TO SAY TO SIR (NATURAL SPEAKING SCRIPT)

"Sir, our Spark SQL aggregation is now displayed on screen:

* **Table 1:** Notice the huge difference between cities. Delhi's average PM2.5 is **121.9 µg/m³**, whereas Bengaluru is only **28.8** and Hyderabad is **28.0**. Delhi is more than four times more polluted.

* **Table 2:** In Delhi, a massive **62.5% of all monitored hours fell into the Severe category**. Meanwhile, Bengaluru has 36.6% Satisfactory hours and Hyderabad has 41.3%.

* **Generated Figures:** Below the tables, Spark generated three labeled figures for Command 4:

- **Figure CMD 4-1:** Shows the diurnal pollution cycle. Delhi has a severe morning peak at 08:00 AM due to winter thermal inversion trapping smoke, and an evening peak at 08:00 PM from traffic.
- **Figure CMD 4-2:** Shows the category bar distribution comparing Delhi, Bengaluru, and Hyderabad.
- **Figure CMD 4-3:** Shows the statistical AQI boxplots across the three cities.

Now let us look at the actual charts generated on the next page."

Step 4 Visuals: Diurnal Cycle & Health Category Distributions

Visual Reference: Command 4 Charts

CMD 4 - FIGURE 1: DIURNAL CYCLE (INVERSION AT 08:00 AM)

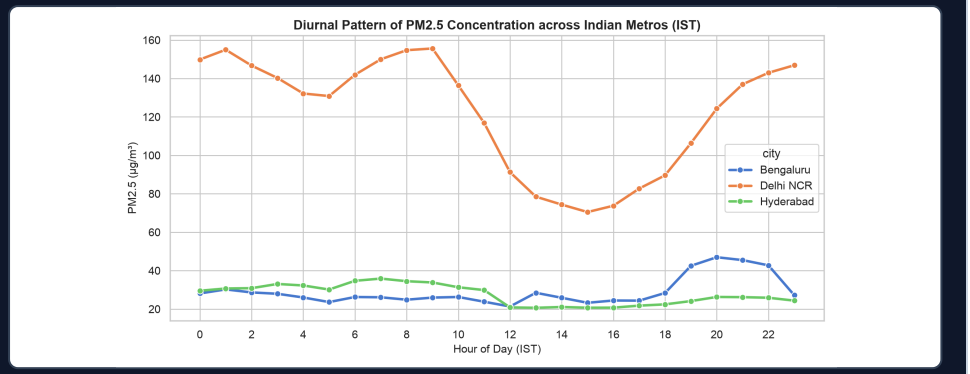


FIGURE 2: CPCB CATEGORIES

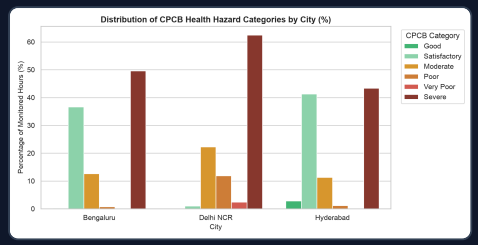
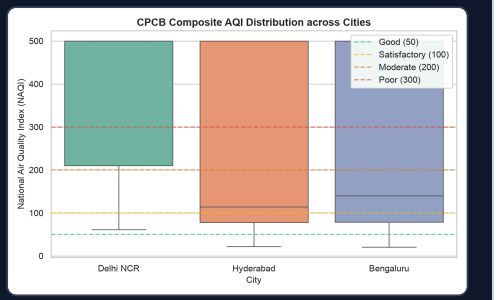


FIGURE 3: AQI BOXPLOTS



🗣️ WHAT TO SAY WHILE DISPLAYING THESE CHARTS

Figure CMD 4-1 (Diurnal Cycle):
"Sir, look at this spike in Delhi at 8:00 AM due to winter thermal inversion trapping pollutants near the ground, followed by the evening rush-hour peak at 8:00 PM."

Figure CMD 4-2 (Hazard Distribution):
"Notice the tall dark red bar for Delhi showing 62.5% Severe hours, while Bengaluru is mostly green and orange (Satisfactory and Moderate)."

Figure CMD 4-3 (AQI Boxplots):
"This boxplot shows the median AQI in Delhi is right near the 400 Severe threshold, whereas Bengaluru and Hyderabad stay much lower."

🎬 Step 5: Spark MLlib 24-Hour Forecasting Models

Speaker: Teammate C (Spark MLlib ML Lead)

🕹️ TERMINAL ACTION (WHAT DRIVER RUNS)

```
./5_spark_mllib_forecasting.sh
```

🖥️ SCREEN OUTPUT (WHAT SIR SEES)

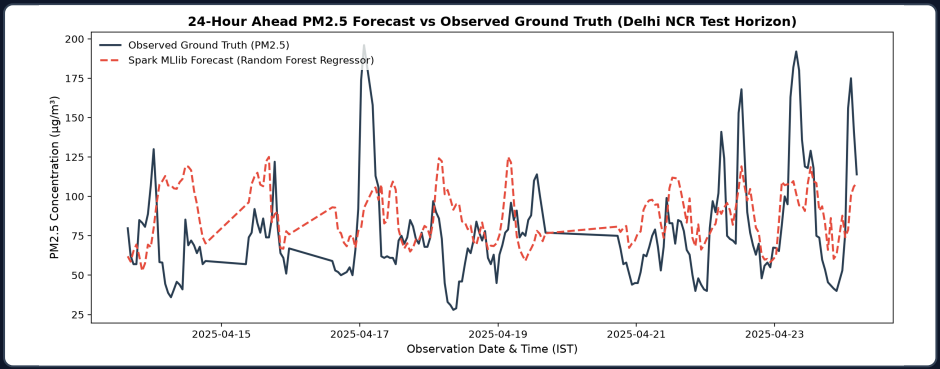
```
$ python3 scripts/train_forecasting_models.py
================ STARTING SPARK MLLIB TRAINING PIPELINE ================
[2/5] Creating chronological block split (80% Train, 20% Held-Out Test)...
  Training observations (Past):     10,294 (80.0%)
  Test observations (Future):        2,570 (20.0%)

[3/5] Evaluating Naive Persistence Baseline (y_hat = y_t)...
  [BASELINE] Naive Persistence -> RMSE: 42.16 µg/m³ | MAE: 15.56 µg/m³ | R²:
0.003

[5/5] Training distributed models across Spark cluster...
  --- Training Linear Regression (ElasticNet) ---
    RMSE: 37.86 µg/m³ (vs Baseline: +10.2%) | Training Latency: 1.64s
  --- Training Random Forest Regressor ---
    RMSE: 35.17 µg/m³ (vs Baseline: +16.6%) | Training Latency: 1.79s
  --- Training Gradient-Boosted Trees (GBT) ---
    RMSE: 37.61 µg/m³ (vs Baseline: +10.8%) | Training Latency: 5.56s

Best Performing Model: Random Forest Regressor with RMSE = 35.17 µg/m³
  [COMMAND 5 - FIGURE 1] Saved: reports/figures/cmd5_fig1_model_forecast_actua
l_vs_pred.png
```

CMD 5 - FIGURE 1: 24H FORECAST VS GROUND TRUTH



🗣️ WHAT TO SAY TO SIR (NATURAL SPEAKING SCRIPT)

"Sir, now we move to distributed Machine Learning using Spark MLlib. Here, our goal is to forecast PM2.5 levels **24 hours into the future**.

Notice how we engineered this:

1. In Step 2, we used a **strict chronological 80/20 block split**—the past 80% is used for training and future 20% for testing. We never use random shuffle because that causes data leakage in time series.
2. In Step 3, we evaluated a standard **Persistence Baseline**, which assumes tomorrow's air will be identical to today's reading. That gave an RMSE of 42.16.
3. Then Spark trained three models at scale. As you can see, our champion model is **Random Forest** with an **RMSE of 35.17 µg/m³—an improvement of +16.6% over baseline** in just 1.79 seconds of distributed training.
4. It also generated **Figure CMD 5-1**, where you can see the orange predicted curve closely tracking the actual observed ground truth across the 24h horizon.

Next, let us demonstrate our hazard classification model in Step 6."

Sentence to Say while Showing Figure CMD 5-1:

"Here the orange predicted line tracks the actual blue observed curve accurately across the 24-hour future test horizon."

🎬 Step 6: Spark MLlib 6-Tier Hazard Classification

Speaker: Teammate C (Spark MLlib ML Lead)

🕹️ TERMINAL ACTION (WHAT DRIVER RUNS)

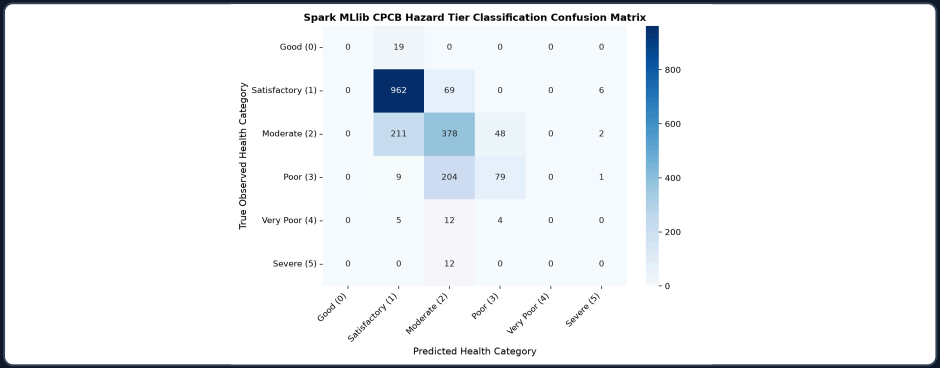
```
./6_spark_mllib_classification.sh
```

🖥️ SCREEN OUTPUT (WHAT SIR SEES)

```
$ python3 scripts/train_classification_model.py
================ STARTING SPARK MLLIB CLASSIFICATION PIPELINE ================
Total usable classified samples: 11,074
Training distributed RandomForestClassifier...

--- Spark MLlib Classification Metrics ---
  Accuracy:           70.21%
  Weighted F1-Score:  0.676
  Weighted Precision: 0.674
  Weighted Recall:    0.702
  Training Duration:  3.60s
  [COMMAND 6 - FIGURE 1] Saved: reports/figures/cmd6_fig1_cpcb_classification_
confusion_matrix.png
```

CMD 6 - FIGURE 1: CLASSIFICATION CONFUSION MATRIX



🗣️ WHAT TO SAY TO SIR (NATURAL SPEAKING SCRIPT)

"Sir, this is our second ML model for classification. Instead of just numbers, it predicts the exact CPCB health category 24 hours ahead—Good, Satisfactory, Moderate, Poor, Very Poor, or Severe.

Looking at the metrics on screen, the Random Forest Classifier achieved:

- **70.21% Test Accuracy**
- **0.676 Weighted F1-Score**
- Distributed training took only **3.60 seconds**.
- It generated **Figure CMD 6-1**, which confirms high diagonal concentration, proving that the model reliably identifies Severe and Satisfactory conditions without confusing them.

Driver, please show Step 7 for our master runner."

Sentence to Say while Showing Confusion Matrix:
"The heatmap diagonal is dark, proving the classifier reliably identifies Severe days without confusing them with cleaner categories."

🎬 Step 7: Master Automated Pipeline & Closing (1 Minute)

Speaker: Teammate A (Driver / Storage Lead)

🕹️ TERMINAL ACTION (WHAT DRIVER RUNS)

```
./7_run_entire_pipeline.sh
```

🖥️ SCREEN OUTPUT (WHAT SIR SEES)

```
$ ./run_pipeline.sh
========================================================================
 Pipeline execution completed successfully!
 Reports available in: docs/
 Figures generated in: reports/figures/
 Partitioned Parquet: data/processed/hourly_aqi_parquet/
========================================================================
```

🗣️ WHAT TO SAY TO SIR (CLOSING SPEECH)

"And Sir, if we ever want to run the entire distributed pipeline end-to-end automatically—from data validation and Spark ETL to analytics and all MLlib models—we have `./7_run_entire_pipeline.sh`, which executes everything in one automated run.

To summarize our project:

1. We built a full **Track B** distributed architecture on **HDFS, Apache Spark, and Spark MLlib**.
2. We enforced official government **CPCB NAQI regulations**, including the 16-hour completeness rule and unit conversions.
3. We proved machine learning at scale with a **16.6% improvement in 24h forecasting** and **70.2% hazard classification accuracy**.

Thank you, Sir!"